Users Connect to Case Manager

In this first section, the connection between the browser and Case Manager will be explained. Case Manager supports high-availability setups that are horizontally scalable. In order to achieve this, multiple Case Manager's instances work side-by-side,Â pushing state out of each Case Manager's instance. This allows instances to fail and integrators to add/remove instances without having to care about state, side-effects and recovery processes.

Browser to Case Managerâ

High availability is implemented in Case Manager, its Load BalancerÂ and Datastore. Case Manager is alsoÂ transparent: users are able to use any Case Manager instance. To achieve this, the Load Balancer is able to track every currently active user that is running the application within the same connection.

In order to understand the details on the implementation of Case Manager, the following schema shows how HTTP Session's state is shared across Case Manager instances, allowing any Case Manager instance to process any request. This allows any instance to fail or be added toÂ theÂ system, scaling the solution horizontally.

Â

Let's get a bit more of detail of each of the steps that are taking within this process:

HTTP Load Balancerâ

The connection between browsers and Case Manager is performed via a stateless Load Balancer that knows where to find all Case Manager instances, thus needing no discovery. Although the load balancer may use sticky sessions, they are not required in order to establish the connection.Â It can also implement dispatch policies such as Round Robbin or Fair dispatch.

This Load Balancer can be either software (Nginx,Â HAProxy) or hardware (Cisco, Barracuda). It also allows monitoring registered instances through an HTTP health check.

Case Manager Instancesâ

Each instance contains the full system with all its four layers:

- A presentation layer consisting of code that runsÂ client-side (inside a web browser).
- A service layer: An entry point to the business layer that enforces authentication and authorization.
- A business layer where all the application logic resides.
- A data layer that is shared across all Case Manager instances, where persistent information is stored.

Also, each of the instances provides a simple REST health check endpoint that reflects the instance health.

To provide high availability and horizontal scalability, the system contemplates that all but one Case Manager's instance can fail. It is also possible to add or remove instances by updating the Load Balancer configuration.

HTTP Session Storageâ

AlthoughÂ **Shiro** currently uses the native HTTP session handled by the servlet container itself, it allows storing sessions in a shared external datastore (e.g. RDBMS, distributed caches andÂ Zookeeper). Using a shared datastore allow all instances to process any request, therefore giving the possibility of adding and removing Case Manager instances as required.

The simplest choice of datastore would be RDBMS / Cassandra, as it is already in use and shared by all Case Manager instances. However, the use of Pulse Zookeeper Cluster would open the possibility of having a Single Sign On for Case Manager and Pulse. This would require both services to be deployed in the same domain, using different context paths or sub-domains.

Learn Moreâ

Continue understanding Case Manager'sÂ **Architecture** by readingÂ how Pulse Talks to Case Manager.